

RISC-V Subset ISA

Single-Cycle Processor Architecture

Design & Reference Document

Document ID: SE-ISA-001
Version: 1.0
Date: June 2026
Author: Prabakaran Elaiyappan

SynaptiqEdge Technologies Pvt Ltd
Chip Design Academy
DPIIT Recognised Startup | Rasipuram, Namakkal, Tamil Nadu

1. Introduction

This document defines a minimal subset of the RISC-V RV32I base integer instruction set, designed specifically for implementing a single-cycle 32-bit processor. The target is educational and research use within the Chip Design Academy curriculum.

The subset is deliberately compact — 29 instructions across 5 encoding formats — yet comprehensive enough to execute real programs: arithmetic, logic, shifts, comparisons, loops, subroutine calls, and unconditional jumps. Memory load/store instructions are excluded in Phase 1 to keep the datapath simple.

All instructions are 32-bit fixed-width and fully compatible with standard RV32I encoding. Any assembler or simulator targeting RV32I can generate binary for this subset without modification.

1.1 Design Goals

- Minimal hardware — single-cycle, no pipeline, no cache, no memory stage
- Full coverage of compute primitives: arithmetic, logic, shift, compare
- Branch and loop support (all six branch conditions)
- Unconditional jump with link register (JAL/JALR) for subroutine calls
- Clean, teachable datapath suitable for RTL implementation in SystemVerilog
- RV32I-compatible binary encoding — no custom opcodes

1.2 Scope & Exclusions

Category	Status	Reason
Arithmetic / Logic / Shift	Included	Core compute capability
Branch / Jump	Included	Loop and control flow
Load / Store (LW/SW)	Excluded — Phase 2	Simplifies datapath (no data mem)
System (ECALL/EBREAK)	Excluded	No OS support required
Multiply/Divide (RV32M)	Excluded	Separate optional extension
Floating Point (F/D)	Excluded	Out of scope for this core

2. Overall Architecture

The processor implements a classical single-cycle Harvard-style architecture. Every instruction completes in exactly one clock cycle. There are no pipeline stages, hazard detection units, or forwarding paths — making it ideal for understanding fundamental CPU operation before tackling pipelining.

2.1 Key Architectural Decisions

- Single-cycle execution: PC updates, register reads, ALU compute, and register write all happen within one clock period.
- Separate instruction memory (ROM) and register file to allow simultaneous access.
- Combinational read ports on the register file; synchronous write on posedge clk.
- All immediate generation is combinational, based on instruction format (I/B/U/J).
- Branch decision = $\text{AND}(\text{Branch_ctrl}, \text{ALU_zero})$. Zero flag from ALU covers BEQ; the control unit inverts or transforms for BNE/BLT/BGE by swapping operands or conditioning on non-zero.
- PC multiplexer selects among four targets: PC+4, branch target, JAL target, JALR target.

2.2 Top-Level Datapath Diagram

The diagram below shows all modules and the signal buses connecting them. Data buses are shown in solid lines; immediate paths in dashed green; control signals in dashed orange; branch/jump targets in dashed red.

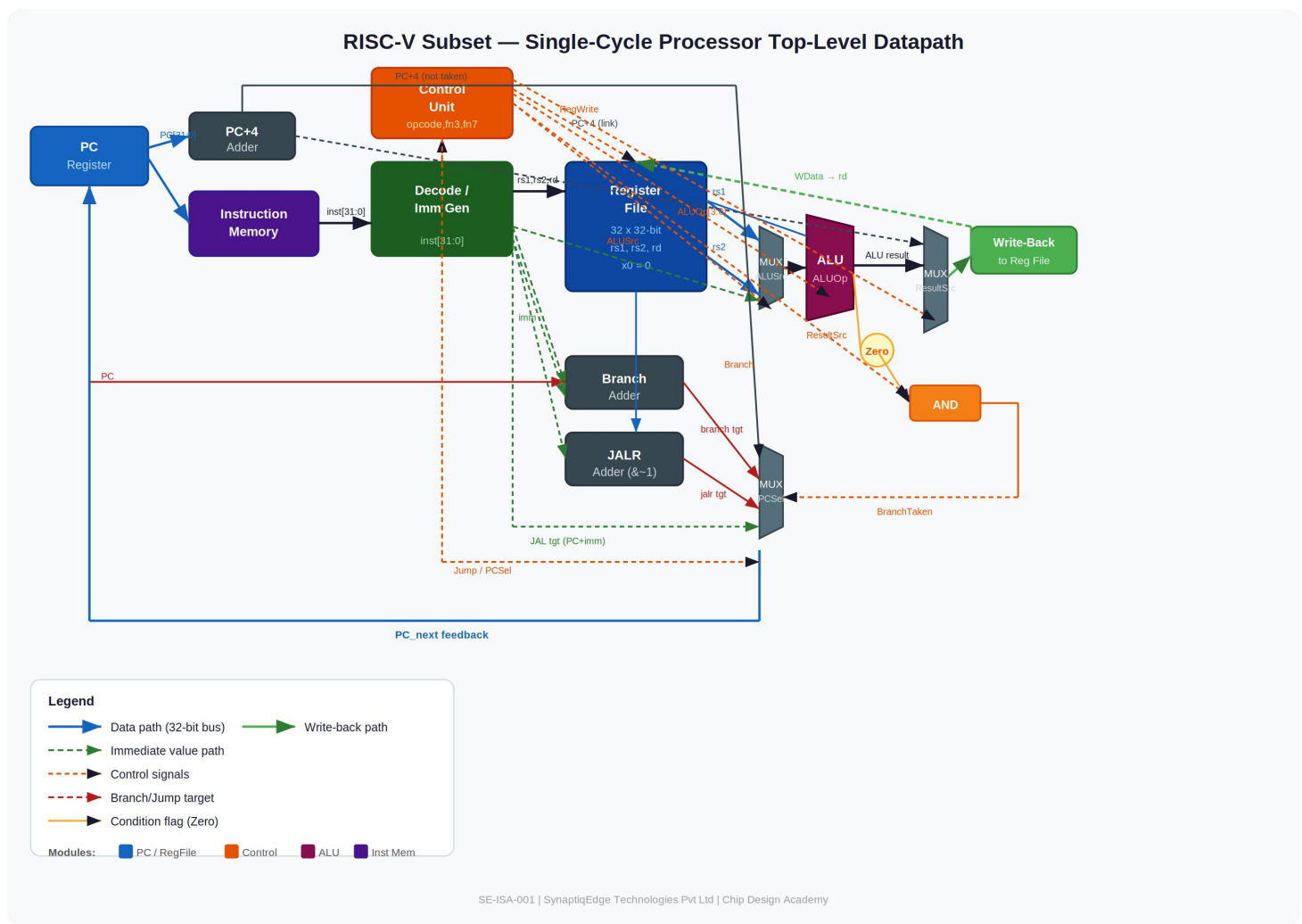


Figure 1 — RISC-V Subset Single-Cycle Processor: Complete Top-Level Datapath

2.3 Module Summary

Module	Type	Function
PC Register	Sequential	Holds current program counter; updates on posedge clk
Instruction Memory	Combinational	ROM; outputs 32-bit instruction word at address PC
Decode / Imm Gen	Combinational	Extracts fields and sign-extends immediate per format
Control Unit	Combinational	Decodes opcode → all datapath control signals
Register File	Sequential (write) / Comb (read)	32 × 32-bit registers; x0 hardwired to 0
ALU	Combinational	Executes ADD/SUB/AND/OR/XOR/SLL/SRL/SRA/SLT/SLTU; outputs result + zero flag
Branch/Jump Logic +	Combinational	Computes targets; selects PC_next

Module	Type	Function
PC Mux		
PC+4 Adder	Combinational	Simple incrementer; provides default next-PC

3. Module Descriptions & Signal Interfaces

Each module below is described with its function, signal interface diagram, port table, and implementation notes.

3.1 PC Register

The Program Counter is a 32-bit register that holds the address of the currently executing instruction. It captures PC_next on every rising edge of clk. A synchronous active-low reset forces PC to 0x00000000.

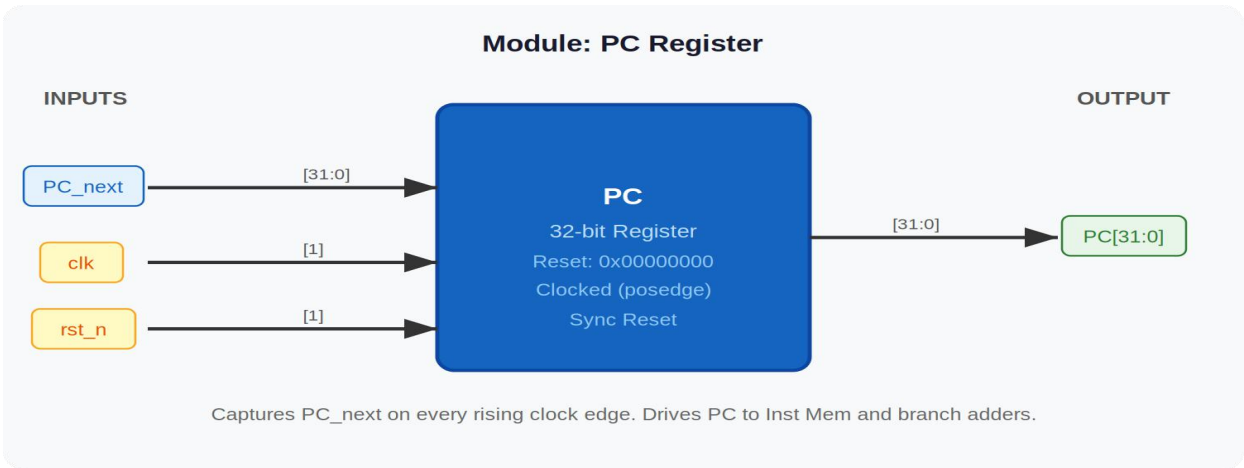


Figure 2 — PC Register: Signal Interface

Port	Width	Direction	Description
PC_next[31:0]	32	Input	Next PC value (from PC Mux)
clk	1	Input	System clock (posedge triggered)
rst_n	1	Input	Active-low synchronous reset
PC[31:0]	32	Output	Current PC → Instruction Memory, Branch Adder

Implementation note: always_ff @(posedge clk) with rst_n check. PC[1:0] must always be 2'b00 (4-byte aligned) — the PC Mux ensures this.

3.2 Instruction Memory

A read-only memory (ROM) that stores the program. It outputs the 32-bit instruction word at the byte address provided by PC. Access is purely combinational (asynchronous read) — there is no clock or enable required.

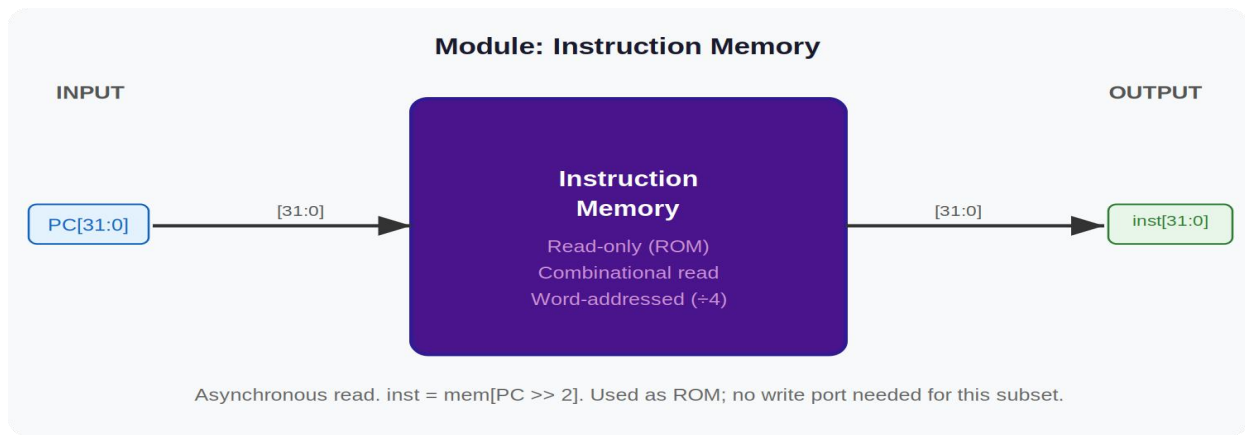


Figure 3 — Instruction Memory: Signal Interface

Port	Width	Direction	Description
PC[31:0]	32	Input	Byte address of instruction to fetch
inst[31:0]	32	Output	32-bit instruction word → Decode/ImmGen, Control

Implementation note: Modelled as a logic array (reg [31:0] mem [0:N-1]). Indexed as $mem[PC[AWIDTH-1:2]]$ (word-addressed). Initialised via \$readmemh in simulation.

3.3 Instruction Decode + Immediate Generator

A purely combinational block that dissects the 32-bit instruction word into its constituent fields and computes a sign-extended 32-bit immediate appropriate to the instruction format. No registers; output valid combinatorially from instruction input.

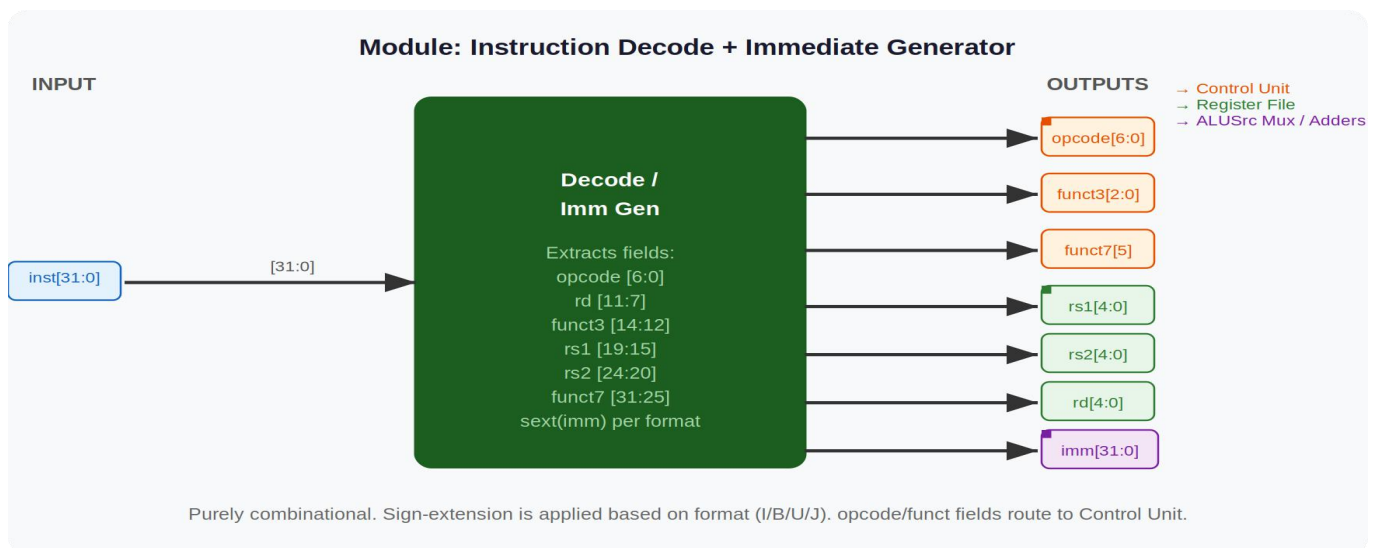


Figure 4 — Instruction Decode / Imm Gen: Signal Interface

Port	Width	Direction	Description
inst[31:0]	32	Input	Raw instruction word from Instruction Memory
opcode[6:0]	7	Output	inst[6:0] → Control Unit
funct3[2:0]	3	Output	inst[14:12] → Control Unit
funct7[5]	1	Output	inst[30] → Control Unit (ADD/SUB, SRL/SRA disambiguation)
rs1[4:0]	5	Output	inst[19:15] → Register File read port A address
rs2[4:0]	5	Output	inst[24:20] → Register File read port B address
rd[4:0]	5	Output	inst[11:7] → Register File write address
imm[31:0]	32	Output	Sign-extended immediate; format selected by opcode

Immediate generation (combinational case statement on opcode):

```

I-type : imm = { {20{inst[31]}}, inst[31:20] }
B-type : imm = { {19{inst[31]}}, inst[31], inst[7], inst[30:25], inst[11:8], 1'b0 }
U-type : imm = { inst[31:12], 12'b0 }
J-type : imm = { {11{inst[31]}}, inst[31], inst[19:12], inst[20], inst[30:21], 1'b0 }

```


3.4 Control Unit

The Control Unit is the decision-making heart of the processor. It takes the instruction type fields (opcode, funct3, funct7[5]) and outputs all datapath control signals. It is a purely combinational block implemented as a priority-encoded case statement on the opcode.

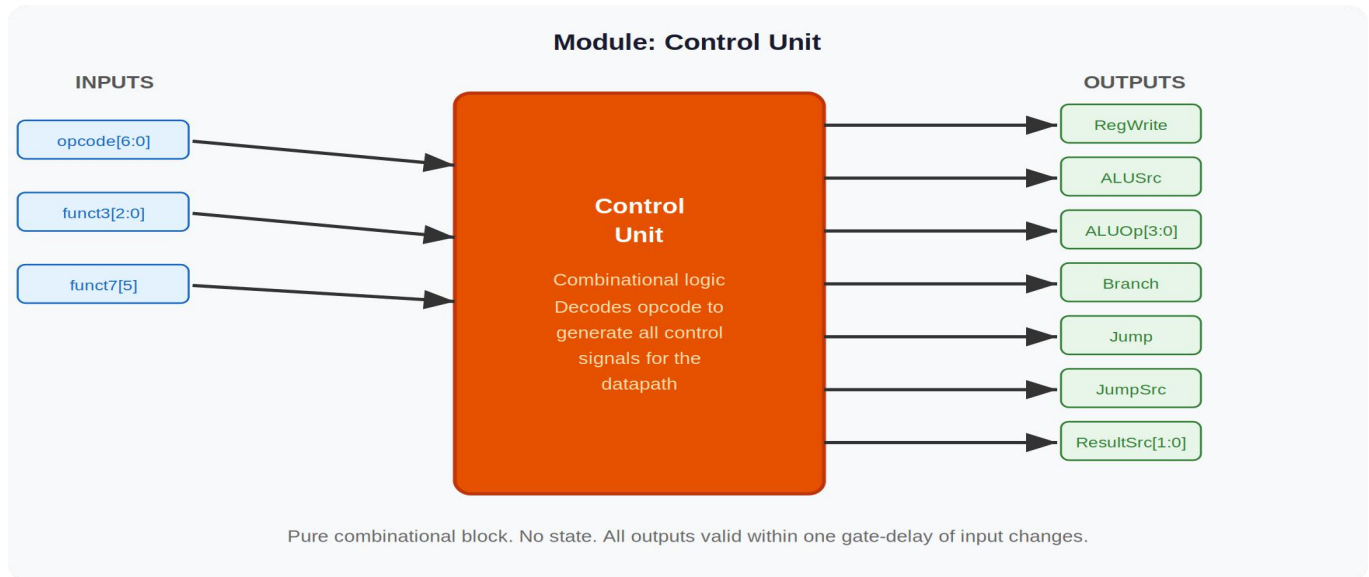


Figure 5 — Control Unit: Signal Interface

Signal	Width	Dir	Description
INPUTS			
opcode[6:0]	7	In	Primary instruction class selector
funct3[2:0]	3	In	Sub-operation; disambiguates branch type and ALU op
funct7[5]	1	In	Distinguishes ADD/SUB and SRL/SRA
OUTPUTS			
RegWrite	1	Out	1 = enable register file write (all except B-type)
ALUSrc	1	Out	0 = B=rs2; 1 = B=immediate (I-type, U-type)
ALUOp[3:0]	4	Out	Selects ALU operation (see Section 5 encoding)
Branch	1	Out	1 = instruction is a conditional branch (B-type)
Jump	1	Out	1 = unconditional jump (JAL or JALR)
JumpSrc	1	Out	0 = JAL (PC-relative); 1 = JALR (register-relative)
ResultSrc[1:0]	2	Out	00=ALU result; 01=PC+4 (link for JAL/JALR); 10=imm<<12 (LUI)

3.5 Register File

The register file holds 32 general-purpose 32-bit registers (x0–x31). x0 is hardwired to 0 — write attempts to x0 are silently suppressed. Two independent asynchronous read ports supply rs1 and rs2 data. One synchronous write port updates rd on the rising clock edge when RegWrite is asserted.

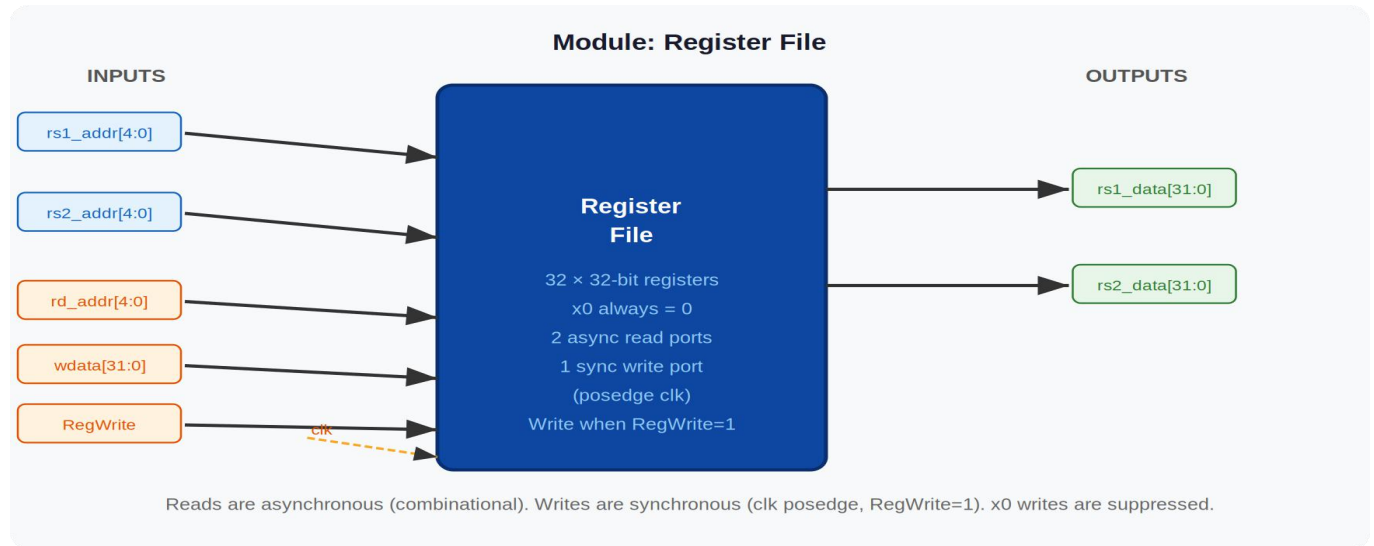


Figure 6 — Register File: Signal Interface

Port	Width	Dir	Description
rs1_addr[4:0]	5	In	Read address A (from Decode)
rs2_addr[4:0]	5	In	Read address B (from Decode)
rd_addr[4:0]	5	In	Write address (from Decode)
wdata[31:0]	32	In	Write data (from Result Mux)
RegWrite	1	In	Write enable (from Control Unit)
clk	1	In	System clock (write on posedge)
rs1_data[31:0]	32	Out	Data for rs1 → ALU port A, JALR Adder
rs2_data[31:0]	32	Out	Data for rs2 → ALUSrc Mux input 0

3.6 ALU (Arithmetic Logic Unit)

The ALU is the computational core. It accepts two 32-bit operands (A = rs1, B = rs2 or immediate) and a 4-bit control code, and produces a 32-bit result plus a single-bit Zero flag used by branch logic.

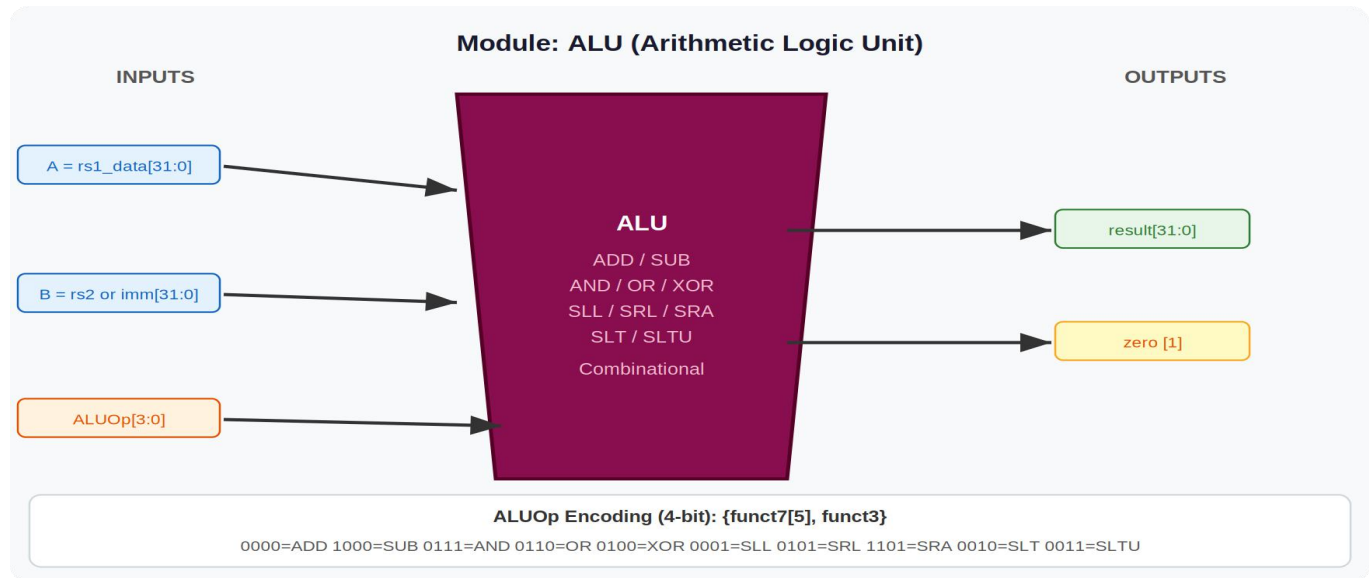


Figure 7 — ALU: Signal Interface & Operation Encoding

Port	Width	Dir	Description
A[31:0]	32	In	Operand A = rs1_data
B[31:0]	32	In	Operand B = rs2_data or sext(imm) via ALUSrc mux
ALUOp[3:0]	4	In	Operation select: {funct7[5], funct3}
result[31:0]	32	Out	ALU output → Result Mux → Register File wdata
zero	1	Out	1 when result == 0; used by branch AND gate

ALUOp[3:0]	funct7[5]	funct3	Operation
0000	0	000	ADD
1000	1	000	SUB
0111	0	111	AND
0110	0	110	OR
0100	0	100	XOR
0001	0	001	SLL (shift left logical)
0101	0	101	SRL (shift right logical)
1101	1	101	SRA (shift right arithmetic)
0010	0	010	SLT (set less than, signed)

ALUOp[3:0]	funct7[5]	funct3	Operation
0011	0	011	SLTU (set less than, unsigned)

3.7 Branch / Jump Logic + PC Multiplexer

This block computes all possible next-PC candidates and selects the correct one based on control signals. It contains two adders (Branch and JALR), an AND gate for branch-taken detection, and a 4-to-1 multiplexer.

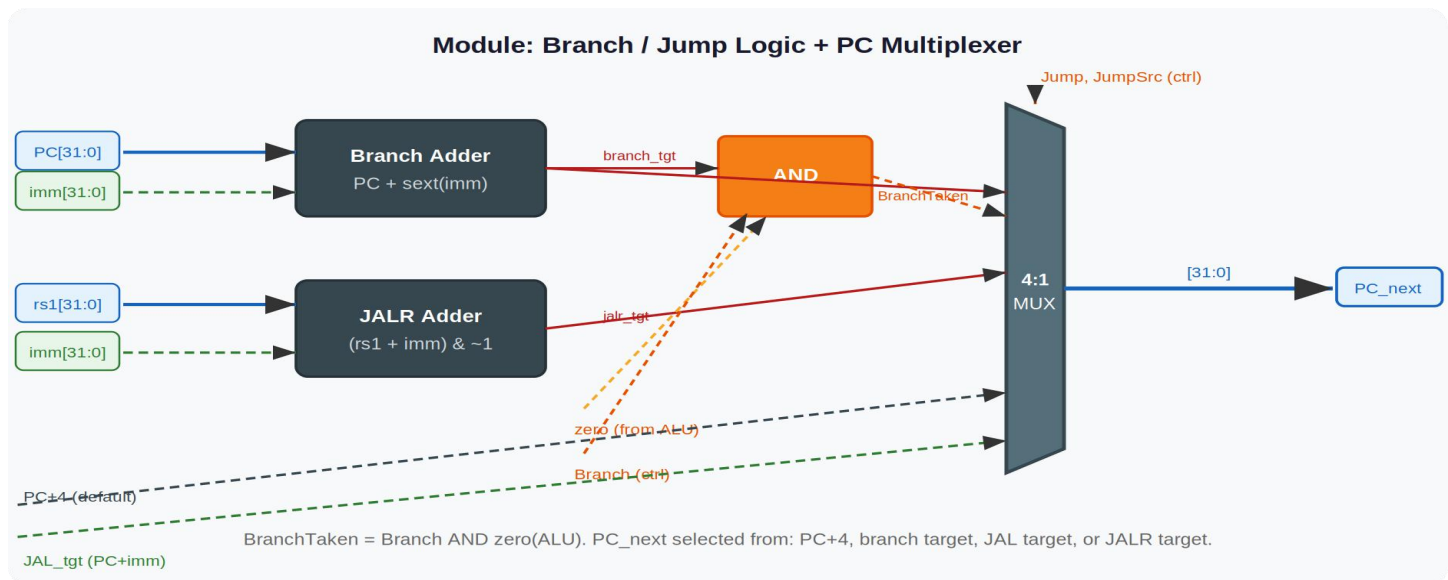


Figure 8 — Branch / Jump Logic + PC Multiplexer: Signal Interface

Port	Width	Dir	Description
PC[31:0]	32	In	Current PC (to Branch Adder and JAL target)
rs1_data[31:0]	32	In	Register rs1 (to JALR Adder)
imm[31:0]	32	In	Sign-extended immediate (to both adders)
PC4[31:0]	32	In	PC+4 (default next PC from PC+4 adder)
zero	1	In	ALU zero flag (branch condition)
Branch	1	In	Control: is this a branch instruction?
Jump	1	In	Control: is this a jump instruction?
JumpSrc	1	In	Control: 0=JAL (PC-rel), 1=JALR (reg-rel)
PC_next[31:0]	32	Out	Selected next PC → PC Register

PC_{next} selection truth table:

Condition	Jump	BranchTaken	PC_next
Normal flow	0	0	PC + 4
Branch taken	0	1	PC + sext(imm) — branch target
JAL	1	JumpSrc=0	PC + sext(imm) — JAL target
JALR	1	JumpSrc=1	(rs1 + sext(imm)) & ~1 — JALR target

4. Instruction Set Reference

4.1 Complete Instruction Summary

#	Mnemonic	Fmt	Operation	Opcode	Category
1	ADD	R	$rd = rs1 + rs2$	0110011	Arithmetic
2	SUB	R	$rd = rs1 - rs2$	0110011	Arithmetic
3	ADDI	I	$rd = rs1 + sext(imm)$	0010011	Arithmetic
4	AND	R	$rd = rs1 \& rs2$	0110011	Logic
5	OR	R	$rd = rs1 rs2$	0110011	Logic
6	XOR	R	$rd = rs1 \wedge rs2$	0110011	Logic
7	ANDI	I	$rd = rs1 \& sext(imm)$	0010011	Logic
8	ORI	I	$rd = rs1 sext(imm)$	0010011	Logic
9	XORI	I	$rd = rs1 \wedge sext(imm)$	0010011	Logic
10	SLL	R	$rd = rs1 \ll rs2[4:0]$	0110011	Shift
11	SRL	R	$rd = rs1 \gg rs2[4:0]$ (logical)	0110011	Shift
12	SRA	R	$rd = rs1 \ggg rs2[4:0]$ (arith)	0110011	Shift
13	SLLI	I	$rd = rs1 \ll shamt$	0010011	Shift
14	SRLI	I	$rd = rs1 \gg shamt$ (logical)	0010011	Shift
15	SRAI	I	$rd = rs1 \ggg shamt$ (arith)	0010011	Shift
16	SLT	R	$rd = (rs1 < rs2) ? 1:0$ (signed)	0110011	Compare
17	SLTU	R	$rd = (rs1 < rs2) ? 1:0$ (unsigned)	0110011	Compare
18	SLTI	I	$rd = (rs1 < sext(imm)) ? 1:0$	0010011	Compare
19	SLTIU	I	$rd = (rs1 < imm) ? 1:0$ (unsigned)	0010011	Compare
20	LUI	U	$rd = imm \ll 12$	0110111	Imm Load
21	AUIPC	U	$rd = PC + (imm \ll 12)$	0010111	PC-relative
22	BEQ	B	if $rs1 == rs2$: $PC += sext(imm)$	1100011	Branch
23	BNE	B	if $rs1 \neq rs2$: $PC += sext(imm)$	1100011	Branch
24	BLT	B	if $rs1 < rs2$ (s): $PC += sext(imm)$	1100011	Branch
25	BGE	B	if $rs1 \geq rs2$ (s): $PC += sext(imm)$	1100011	Branch
26	BLTU	B	if $rs1 < rs2$ (u): $PC += sext(imm)$	1100011	Branch
27	BGEU	B	if $rs1 \geq rs2$ (u): $PC += sext(imm)$	1100011	Branch
28	JAL	J	$rd = PC + 4$; $PC += sext(imm)$	1101111	Jump
29	JALR	I	$rd = PC + 4$; $PC = (rs1 + sext(imm)) \& \sim 1$	1100111	Jump

4.2 Register ABI Reference

Register	ABI Name	Role / Convention
x0	zero	Hardwired 0 — reads always return 0, writes silently ignored
x1	ra	Return address — saved by JAL/JALR
x2	sp	Stack pointer (unused in Phase 1 — no memory)
x5–x7	t0–t2	Temporary registers — caller-saved
x8–x9	s0–s1	Saved registers — callee-saved
x10–x11	a0–a1	Function arguments / return values
x12–x17	a2–a7	Additional function arguments
x18–x27	s2–s11	Additional saved registers
x28–x31	t3–t6	Additional temporaries — caller-saved

4.3 Pseudo-Instruction Reference

Pseudo	Expands To	Effect
NOP	ADDI x0, x0, 0	No operation
MV rd, rs	ADDI rd, rs, 0	Copy register
LI rd, imm	LUI + ADDI	Load 32-bit immediate
NEG rd, rs	SUB rd, x0, rs	Two's complement negation
NOT rd, rs	XORI rd, rs, -1	Bitwise NOT
BEQZ rs, L	BEQ rs, x0, L	Branch if zero
BNEZ rs, L	BNE rs, x0, L	Branch if non-zero
BLTZ rs, L	BLT rs, x0, L	Branch if negative
BGEZ rs, L	BGE rs, x0, L	Branch if non-negative
J label	JAL x0, label	Unconditional jump (discard link)
RET	JALR x0, ra, 0	Return from subroutine
CALL label	JAL ra, label	Call subroutine (save return address)

5. Revision History

Version	Date	Author	Change
1.0	June 2026	Prabakaran Elaiyappan	Initial release — 29-instruction subset with full architecture documentation